Lab 2 - DDIM and LoRA

Image and Video Generation

應數所　王元亨　315652003

Abstract

本實驗包含兩個主題。第一部分實作Denoising Diffusion Implicit Models（DDIM），先在二維Swiss Roll 資料上驗證反向生成流程，再將相同概念套用至預訓練影像擴散模型，分析推論步數與隨機性參數η 對Fréchet Inception Distance（FID）的影響。第二部分使用Low-Rank Adaptation（LoRA）微調Stable Diffusion，分別學習Naruto 視覺風格，以及透過DreamBooth with LoRA 學習特定貓咪的身份。

Contents

1 Task 1：Denoising Diffusion Implicit Models **2**
- 1.1 Task 1-1：2D Swiss Roll . . . 2
  - 1.1.1 實驗目的與前置DDPM 元件 . . . 2
  - 1.1.2 DDIM 單步反向取樣 . . . 2
  - 1.1.3 DDIM inference loop . . . 3
  - 1.1.4 Swiss Roll 評估結果 . . . 3
- 1.2 Task 1-2：DDIM Image Generation . . . 3
  - 1.2.1 Inference timestep 設定 . . . 3
  - 1.2.2 DDIMScheduler 單步更新 . . . 4
  - 1.2.3 實驗設定 . . . 4
  - 1.2.4 FID 結果與討論 . . . 4

2 Task 2：Training Custom LoRA Models **6**
- 2.1 方法簡介 . . . 6
- 2.2 Task 2-1：Naruto Style LoRA . . . 6
  - 2.2.1 資料集與訓練設定 . . . 6
  - 2.2.2 生成結果 . . . 6
- 2.3 Task 2-2：DreamBooth with LoRA . . . 7
  - 2.3.1 資料集與訓練設定 . . . 7
  - 2.3.2 生成結果 . . . 8

3 結論 **8**

1 Task 1：Denoising Diffusion Implicit Models

1.1 Task 1-1：2D Swiss Roll

1.1.1 實驗目的與前置DDPM 元件

令x_0 爲眞實資料、$\epsilon \sim \mathcal{N}(0, I)$，DDPM 的前向加噪可直接寫爲

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s. \tag{1}$$

程式中的`q_sample` 依此公式產生x_t；`compute_loss` 隨機抽取timestep，讓網路預測加入的噪聲，並以

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0,t,\epsilon}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2\right] \tag{2}$$

作爲訓練目標。二維噪聲預測器由多層`TimeLinear` 組成，將sinusoidal timestep embedding 轉換後乘入各層特徵；最後一層投影回二維噪聲。`p_sample` 實作單步DDPM posterior sampling，`p_sample_loop` 則從高斯噪聲開始逐步走完所有訓練timesteps。這些元件提供可用的噪聲預測網路，之後只更換反向取樣規則爲DDIM。

1.1.2 DDIM 單步反向取樣

在當前timestep t，由網路得到$\epsilon_\theta(x_t, t)$，首先估計乾淨資料：

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\,\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}. \tag{3}$$

接著以η 控制每一步的隨機程度：

$$\sigma_t = \eta \sqrt{\frac{1 - \bar{\alpha}_{t'}}{1 - \bar{\alpha}_t}} \sqrt{1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t'}}}, \tag{4}$$

其中t' 是下一個較小的inference timestep。更新式爲

$$x_{t'} = \sqrt{\bar{\alpha}_{t'}}\hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t'} - \sigma_t^2}\, epsilon_\theta(x_t, t) + \sigma_t z, \qquad z \sim \mathcal{N}(0, I). \tag{5}$$

當$\eta = 0$ 時，$\sigma_t = 0$，在固定初始噪聲的條件下更新爲deterministic DDIM。

Listing 1: `ddim_p_sample`

```
alpha_t = extract(self.var_scheduler.alphas_cumprod, t, xt)
alpha_prev = (
    extract(self.var_scheduler.alphas_cumprod, t_prev, xt)
    if t_prev >= 0 else torch.ones_like(alpha_t)
)
eps_theta = self.network(xt, t)
x0_pred = (xt - torch.sqrt(1 - alpha_t) * eps_theta) / torch.sqrt(alpha_t)

sigma_t = eta * torch.sqrt((1 - alpha_prev) / (1 - alpha_t)) \
              * torch.sqrt(1 - alpha_t / alpha_prev)
direction = torch.sqrt(1 - alpha_prev - sigma_t ** 2) * eps_theta
noise = torch.randn_like(xt) if eta > 0 else torch.zeros_like(xt)
x_prev = torch.sqrt(alpha_prev) * x0_pred + direction + sigma_t * noise
```

1.1.3 DDIM inference loop

`ddim_p_sample_loop` 先由訓練步數與指定推論步數計算

$$r = \left\lfloor \frac{T_{\text{train}}}{T_{\text{infer}}} \right\rfloor, \tag{6}$$

再建立由大到小排列的timestep 子集合。取樣從標準高斯噪聲開始，每次呼叫單步DDIM 更新，直到$t' < 0$ 時令$\bar{\alpha}_{t'} = 1$，得到最終x_0。

Listing 2: `ddim_p_sample_loop` timestep

```
step_ratio = self.var_scheduler.num_train_timesteps // num_inference_timesteps
timesteps = (
    (np.arange(num_inference_timesteps) * step_ratio)
    .round()[::-1].copy().astype(np.int64)
)
prev_timesteps = timesteps - step_ratio

xt = torch.randn(shape).to(self.device)
for t, t_prev in zip(timesteps, prev_timesteps):
    xt = self.ddim_p_sample(xt, t, t_prev, eta=eta)
```

1.1.4 Swiss Roll 評估結果

圖 1 顯示目標分布與DDIM 生成樣本。多數生成點能沿著兩圈Swiss Roll 排列，僅有少量點落在曲線之外。Chamfer Distance 爲18.4572。

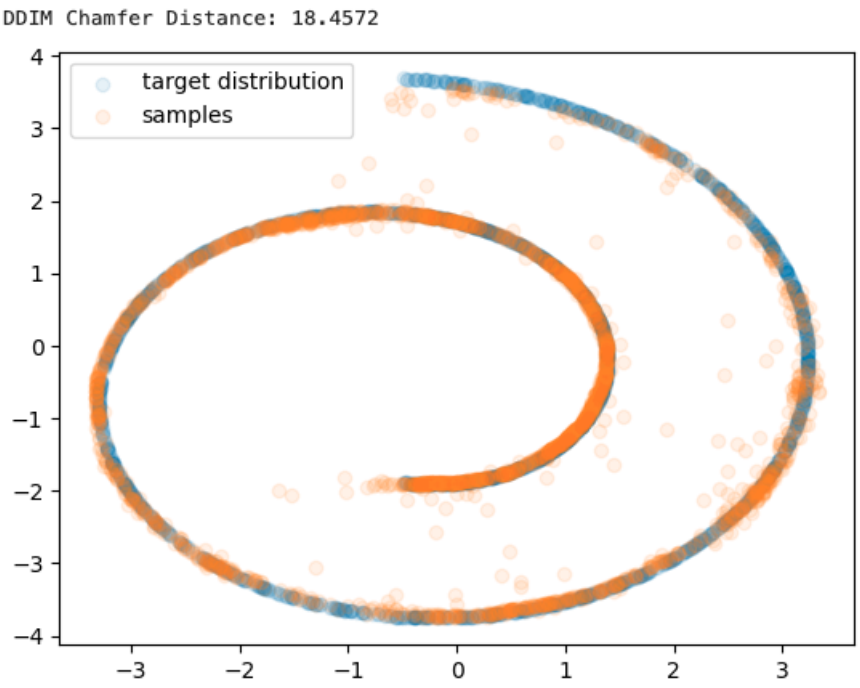


Figure 1: DDIM 在Swiss Roll 上的生成結果與Chamfer Distance。

1.2 Task 1-2：DDIM Image Generation

1.2.1 Inference timestep 設定

影像實驗沿用Lab 1 noise-prediction checkpoint 的beta schedule，避免訓練與推論的$\bar{\alpha}_t$ 不一

致。`set_inference_timesteps` 依指定步數建立訓練timestep 的子集合，並記錄相鄰inference steps 的間距。

Listing 3: `DDIMScheduler.set_inference_timesteps`

```
step_ratio = self.num_train_timesteps // num_inference_timesteps
timesteps = (
    (np.arange(num_inference_timesteps) * step_ratio)
    .round()[::-1].copy().astype(np.int64)
)
self.timesteps = torch.from_numpy(timesteps)
self._ddim_step_ratio = step_ratio
```

1.2.2 DDIMScheduler 單步更新

先計算$t' = t - r$，再由ϵ_θ 推回$\hat{x}_0$；因影像已正規化至$[-1, 1]$，程式將$\hat{x}_0$ clamp 至合法範圍以降低數值誤差。最後依前述DDIM 方程式組合predicted sample、direction term 與隨機項。$\eta = 0$ 時不建立額外噪聲；$\eta > 0$ 時則加入$\sigma_t z$。

Listing 4: `DDIMScheduler.step`

```
t_prev = t - self._ddim_step_ratio
alpha_t = self._get_teeth(self.alphas_cumprod, t_tensor)
alpha_prev = (
    self._get_teeth(self.alphas_cumprod, t_prev_tensor)
    if t_prev >= 0 else torch.ones_like(alpha_t)
)
x0_pred = (x_t - torch.sqrt(1 - alpha_t) * eps_theta) / torch.sqrt(alpha_t)
x0_pred = x0_pred.clamp(-1.0, 1.0)

sigma_t = self.eta * torch.sqrt((1 - alpha_prev) / (1 - alpha_t)) \
                   * torch.sqrt(1 - alpha_t / alpha_prev)
direction = torch.sqrt(1 - alpha_prev - sigma_t ** 2) * eps_theta
noise = torch.randn_like(x_t) if self.eta > 0 else torch.zeros_like(x_t)
sample_prev = torch.sqrt(alpha_prev) * x0_pred + direction + sigma_t * noise
```

1.2.3 實驗設定

每一組設定生成500 張影像，並以AFHQ evaluation set 爲reference 計算FID。除DDIM inference steps 與η 外，其餘條件保持一致。

1.2.4 FID 結果與討論

Table 1: 不同DDIM steps 與η 的FID。粗體爲全體最佳結果。

η	DDIM steps				
	10	20	50	100	1000
0.0	39.52	29.93	19.12	12.16	41.93
0.2	38.85	30.29	17.01	10.71	27.71
0.5	35.62	25.83	16.73	12.12	**6.82**
1.0	54.74	34.79	21.86	15.06	7.40

由表 1 可觀察到三點。第一，在$\eta = 0.5$ 與$\eta = 1.0$ 時，增加inference steps 能穩定改善FID；例如$\eta = 0.5$ 由10 steps 的35.62 降至1000 steps 的6.82。第二，step 數很少時過大的隨機性會傷害品質，10 steps、$\eta = 1.0$ 的FID 54.74 爲最差結果。第三，$\eta = 0$ 與$\eta = 0.2$ 在1000 steps 出現反常退化，顯示更多步數不必然保證較佳結果；可能原因包括deterministic trajectory 對checkpoint 誤差的累積、有限樣本造成的FID 波動，或不同隨機起點造成的估計差異。若要驗證此現象，可固定seed 並重複生成多組500 張樣本，回報平均值與標準差。

圖 2 選擇最差與最佳設定的單張輸出作定性比較。低FID 組合的紋理與輪廓較清楚，但單張影像僅作視覺輔助，主要結論仍以500 張樣本計算的FID 爲準。



(a) 10 steps，$\eta = 1.0$，FID 54.74。 (b) 1000 steps，$\eta = 0.5$，FID 6.82。

Figure 2: 不同DDIM 設定的代表性生成影像。

2 Task 2：Training Custom LoRA Models

2.1 方法簡介

LoRA 將原始權重W 凍結，只訓練低秩更新：

$$W' = W + \Delta W, \qquad \Delta W = BA, \tag{7}$$

其中秩r 遠小於原權重維度，因此可用較少參數讓預訓練模型適應新風格或身份。本實驗以Stable Diffusion v1.4 爲base model，分別訓練style LoRA 與DreamBooth LoRA。

2.2 Task 2-1：Naruto Style LoRA

2.2.1 資料集與訓練設定

本實驗使用Hugging Face 上的lambdalabs/naruto-blip-captions。資料集由Naruto 動畫影像及BLIP 產生的文字描述組成，適合用來學習角色線條、色彩與動畫式陰影。圖 3 左側爲資料範例，右側示範相同prompt 在LoRA 前後的外觀差異。

Table 2: Style LoRA 訓練設定。

項目	設定
Base model	`CompVis/stable-diffusion-v1-4`
Dataset	`lambdalabs/naruto-blip-captions`
Resolution	512×512
Batch size	1
Epochs	100
Learning rate	1×10^{-4}
Scheduler	constant
Seed	42



Figure 3: Naruto LoRA 的代表性訓練資料，以及載入LoRA 前後的生成差異。圖片來源爲課程提供範例。

2.2.2 生成結果

爲避免prompt 本身直接指定動畫風格，以下prompts 不包含“Naruto style” 或“anime style”。結果顯示人物、動物與動作場景都呈現較一致的動畫線條、平面色彩與陰影，表示LoRA 學到可轉移的視覺風格，而非只記住單一角色。



(a) `a man with sunglasses`



(b) `a young swordsman fighting in a bamboo forest, dynamic action pose`



(c) `a white fox running across a snowy mountain`

Figure 4: Style LoRA 的生成結果

LoRA 權重 Style LoRA 權重上傳於GitHub。

2.3 Task 2-2：DreamBooth with LoRA

2.3.1 資料集與訓練設定

本實驗使用課程repository 提供的四張橘色長毛貓影像，並以稀有識別詞`sks cat` 綁定該身份。四張訓練圖具有不同視角、姿勢與室內背景，可幫助模型區分貓咪的固定外觀與可變場景。

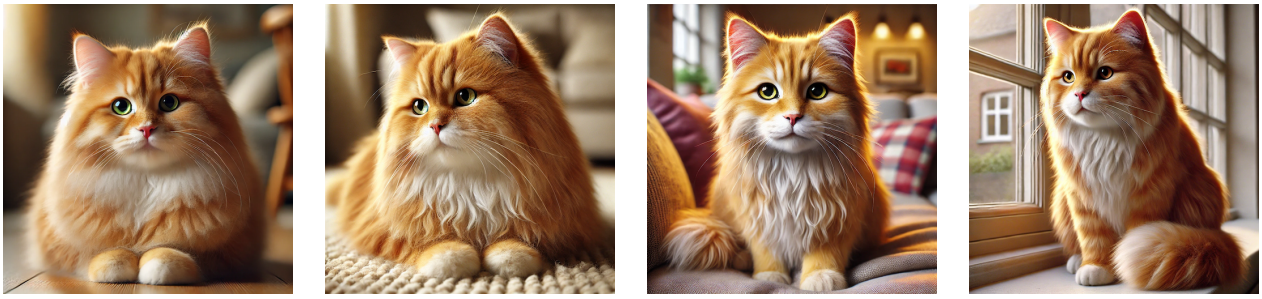


Figure 5: DreamBooth LoRA 使用的四張身份訓練影像。

Table 3: DreamBooth LoRA 訓練設定。

項目	設定
Base model	`CompVis/stable-diffusion-v1-4`
Instance prompt	`a photo of a sks cat`
Resolution	512×512
Batch size	1
Maximum training steps	500
Learning rate	1×10^{-4}
Scheduler	constant
Seed	0

2.3.2 生成結果

前兩個推論prompt 保留`sks cat` 識別詞，並改變成訓練資料中沒有的海灘與桶子場景。圖 6 前兩張影像仍保留橘色長毛、白色胸毛及相近臉部特徵，顯示模型能在場景改變時維持主要身份特徵。第三張使用不含`sks cat` 的prompt `a man with sunglasses`，結果生成一般男性而非目標貓咪，可作爲負向對照，說明身份識別詞是觸發DreamBooth 個體概念的關鍵。另一方面，毛色飽和度與臉部細節仍會受prompt、seed 與base model prior 影響，因此身份重建並非像素級複製。



(a) `a photo of sks cat on the beach` (b) `a photo of sks cat in a bucket` (c) `a man with sunglasses`

Figure 6: DreamBooth LoRA 的生成結果

LoRA 權重 DreamBooth LoRA 權重上傳於GitHub。

3 結論

本實驗完成二維與影像版本的DDIM noise-prediction sampling。Swiss Roll 的Chamfer Distance 爲18.4572；影像實驗的最佳組合爲1000 steps、$\eta = 0.5$，FID 爲6.82。結果顯示減少反向步數可大幅加速生成，但步數過少時品質會下降；η 的適當隨機性也會影響生成分布。LoRA 實驗則顯示，少量可訓練參數即可讓Stable Diffusion 學習Naruto 視覺風格；搭配DreamBooth 與識別詞後，也能在不同場景中重現特定貓咪身份。

References

[1] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," *International Conference on Learning Representations*, 2021.

[2] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," *Advances in Neural Information Processing Systems*, 2020.

[3] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," *International Conference on Learning Representations*, 2022.

[4] N. Ruiz et al., "DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation," *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.

[5] Lambda Labs, "Naruto BLIP Captions," Hugging Face Datasets. https://huggingface.co/datasets/lambdalabs/naruto-blip-captions